Data-Centric Systems

Lecture 1: Introduction and Database Architecture



Dr. Yatish Bathla

Ph.D. (Information Technology), Obuda University, Budapest

Outcomes and agenda

1

Explain
why organisations build systems around data

2

Describe
how networks carry data (layers, client-server)

3

Name
the parts of a DBMS and what each does

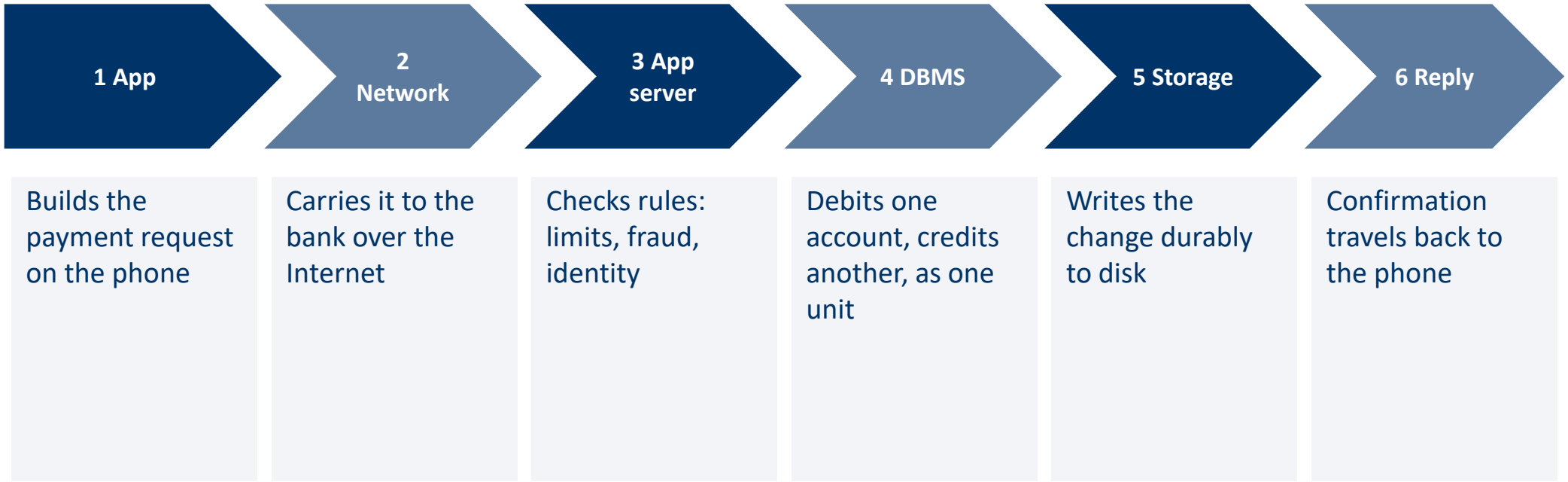
4

Draw
a three-tier application and trace a request

KEY IDEA

Learning focus: integrated systems orientation. Where does each idea sit in the path of a request?

Hook: what happens when you tap "Pay"?



KEY IDEA **One tap touches an application, a network, a DBMS and storage. This lecture covers each stage.**

Data, information, knowledge

Data

A recorded fact

Balance = 5,000

Information

Data in context

Customer 101 has 5,000 in a savings account

Knowledge

Information that drives action

The account is below the minimum, so charge a fee or notify the customer

KEY IDEA

Data-centric systems exist to turn stored data into decisions and actions.

Application-centric vs data-centric

Application-centric	Older pattern

- Each application owns its own files
- Same data copied into many systems
- Changing a format means changing every program
- Hard to answer questions that cross applications

Data-centric	Our focus

- Data is managed once, shared by many applications
- The DBMS enforces rules and protects the data
- Applications can change without moving the data
- Cross-cutting questions are one query away

KEY IDEA **In a data-centric system the data outlives any single application.**

Where data-centric systems run

Industry	Typical data	Example workload
Finance	Accounts, transactions, positions	Transfers, settlement, risk reports
Insurance	Policies, claims, customers	Quotes, claim approval, renewals
Healthcare	Patient records, orders, results	Admissions, lab results, billing
Aerospace	Maintenance logs, parts, telemetry	Inspection schedules, parts tracking
Automotive	Vehicle telemetry, supply chain	Diagnostics, recalls, logistics
Telecom	Call and usage records, subscribers	Billing, network planning

KEY IDEA	**Different industries, the same pattern: many users, shared data, strict correctness.**

Why not just use files? Eight problems

Redundancy
Same data stored in many places

Inconsistency
Copies disagree after an update

Durability
Changes are permanent, even if the system crashes

Isolation
Data scattered across formats

Integrity
Rules buried in application code

Atomicity
A failure leaves half an update

Concurrency
Two users overwrite each other

Security
Permissions only per file

KEY IDEA A DBMS removes these problems (SKS 1.2 lists seven; we split the first into two).

Worked example: the lost update

Time	Clerk A	Clerk B	Balance in file
t1	Reads balance		5,000
t2		Reads balance	5,000
t3	Adds 1,000 and writes 6,000		6,000
t4		Subtracts 500 and writes 4,500	4,500
Result	Deposit of 1,000 is lost		Should be 5,500

KEY IDEA **Two correct programs can still produce a wrong result; the DBMS prevents this.**

The data lifecycle

Capture	Transport	Store	Process	Present
Forms, sensors, transactions	Networks move it to servers	DBMS keeps it safely	Queries, analysis, statistics	Reports, dashboards, apps

KEY IDEA **Lectures 1 and 2 follow this path: transport and store today, then structure and summarise.**

The network that carries the data

Nuts-and-bolts view — Kurose 1.1.1

- Billions of devices: hosts or end systems
- Links (fibre, copper, radio) joined by packet switches and routers
- Everything runs through agreed protocols

Services view — Kurose 1.1.2

- An infrastructure that gives applications a service
- Applications are distributed: they run on many hosts and exchange messages
- A protocol defines message format, order and actions

KEY IDEA **A protocol is an agreed set of rules for exchanging messages. SQL clients and servers use one too.**

Layers: how a query travels

Application
SQL request, HTTP: the message the program means

Transport
TCP: reliable delivery between processes (port 5432)

Network
IP: addressing and routing across networks

Link
Ethernet, Wi-Fi: one hop at a time

Physical
Bits as signals on a cable or radio

What to remember

- Each layer offers a service to the one above and uses the one below
- A SQL query is application-layer data; TCP carries it, IP routes it
- Tools map to layers: ping tests the network layer, a port test reaches transport
- Lab: we capture and inspect these layers

KEY IDEA **Layering lets each part change without breaking the others.**

Client-server and peer-to-peer

Client-server	Used by DBMSs

- Server: always on, fixed address, waits for requests
- Clients: initiate contact, may come and go
- Example: an app queries a database server

Peer-to-peer	Contrast

- Peers act as both client and server
- No always-on central server
- Example: file sharing; rare for enterprise databases

KEY IDEA	**Database systems are client-server: the DBMS waits, the application asks.**

Checkpoint

- Name two problems of storing the same customer data in several files.
- A SQL query travels from client to DBMS. Which layer holds the SQL text, and which protocol makes delivery reliable?
- Who is the client and who is the server when you open a bank app?
- Which of the eight problems is the lost update?

KEY IDEA	**If you can answer all four, you are ready for database architecture.**

Three levels of data abstraction

View level
What each user or application sees: parts of the data, customised

Logical level
What data is stored and how it relates: tables, attributes, keys

Physical level
How data is stored: files, pages, indexes on disk

Who works at each level

- View level: many views per database; hides detail and adds security
- Logical level: the database administrator and designers work here
- Physical level: the storage manager decides file layout and indexes
- Higher levels do not depend on lower-level detail

KEY IDEA **Hiding detail at each level is what lets the data and the programs evolve separately.**

Data independence

Physical data independence — Storage can change

- Move data to new disks, add an index, change file layout
- Logical schema and programs keep working
- Only performance changes

Logical data independence — Schema can grow

- Add a column or a table
- Existing views and programs keep working
- Harder to achieve fully than physical independence

KEY IDEA

Independence = change one level without rewriting the levels above it.

Database languages: DDL and DML

SQL

```
-- DDL: define structure
CREATE TABLE account (
  acct_no  INTEGER PRIMARY KEY,
  owner    VARCHAR(40) NOT NULL,
  balance  NUMERIC(12,2) CHECK (balance >= 0)
);

-- DML: query and modify
SELECT owner, balance FROM account
  WHERE balance > 10000;
UPDATE account SET balance = balance - 500
  WHERE acct_no = 101;
```

Two sub-languages

- DDL defines schema and constraints
- DML queries and changes data
- SQL is declarative: say what you want, not how to get it
- The DBMS chooses the plan

KEY IDEA **DDL constraints (PRIMARY KEY, CHECK) are enforced by the DBMS for every application.**

Inside the DBMS engine

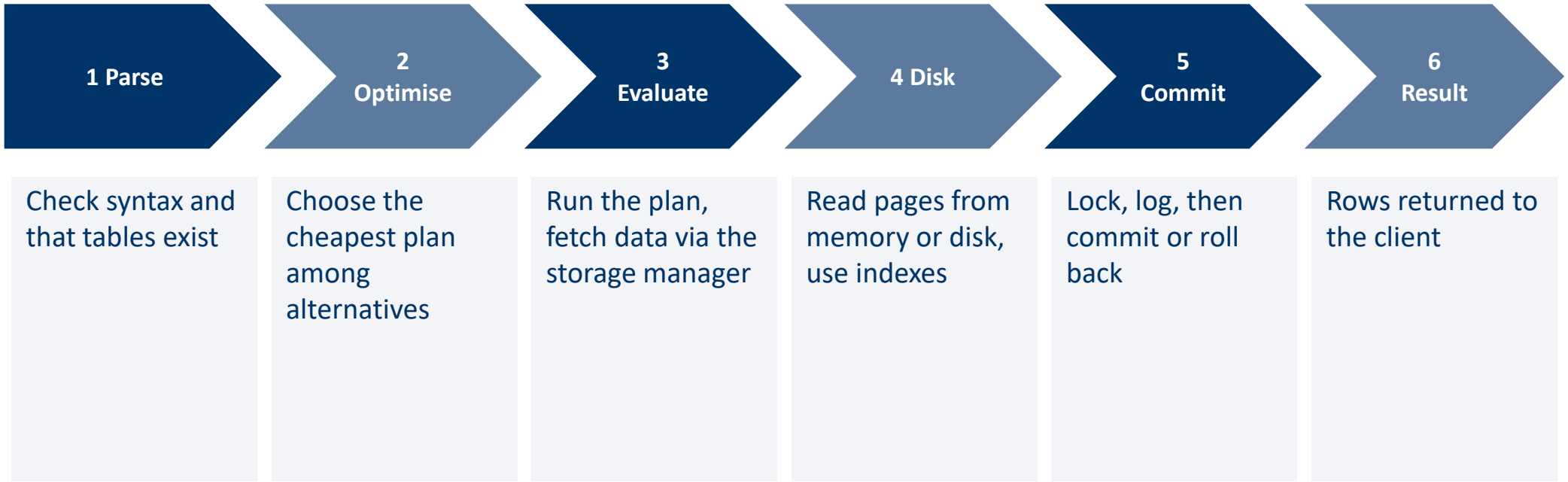
Storage manager
Files, buffer manager, indexes
Stores and retrieves data on disk efficiently

Query processor
Parser, optimiser, evaluation engine
Turns SQL into an efficient plan and runs it

Transaction manager
Concurrency control, recovery
Keeps results correct under failures and concurrent users

KEY IDEA **Three cooperating components: store it, query it, and keep it correct.**

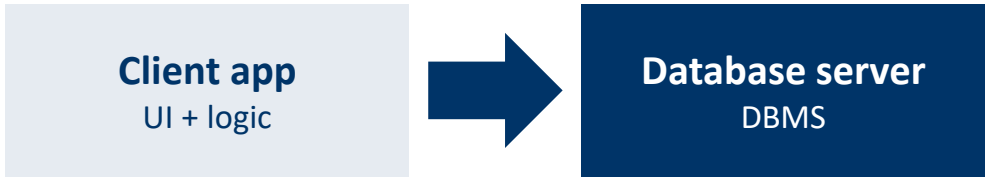
Following one query through the engine



KEY IDEA **Every SQL statement passes through all three components.**

Application architecture: two-tier vs three-tier

Two-tier — Earlier generation



- Application code runs on every client
- Clients send SQL straight to the DBMS
- Hard to secure; each client needs database access
- Updating logic means updating every client

Three-tier — Modern applications



- Clients never touch the database
- Business logic lives in one place
- Better security and scalability than two-tier
- Standard for web and mobile systems

KEY IDEA **Three-tier puts a controlled layer between users and the data (SKS 1.7).**

Users, administrators and deployment

Role or option	What they do or what it means
Naive users	Use forms and apps: tellers, web shoppers
Application programmers	Write the programs that talk to the DBMS
Analysts	Query data directly for decisions: SQL, BI tools
Database administrator (DBA)	Schema, access control, backup, tuning, availability
Centralised vs client-server	One machine, or DBMS on a server with many clients
Distributed	Data spread over sites, connected by a network

KEY IDEA	**Networks and databases meet at deployment: distributed systems.**

Putting it together: the Pay request

Stage	Component	What happens
Tap Pay	Application (client)	Builds an HTTPS request with payment details
Delivery	Network layers	TCP and IP carry the message to the bank
Business rules	Application server (tier 2)	Checks limits, identity and fraud rules
Parse and optimise	Query processor	SQL is checked and a plan is chosen
Debit and credit	Transaction manager	Both updates commit together, or neither does
Durable write	Storage manager and disk	Log and data pages are written safely
Confirmation	Network and client	Reply returns and the app shows success

KEY IDEA	**Integrated systems orientation = seeing the whole path, not one part.**

Summary and exit question

WRAP-UP

Exit question

- A lost-update error appears in a bank system. Which component is responsible for preventing it, and at which layer of the stack does the SQL text live?
- Next: Lecture 2 – how data is structured (relational theory) and summarised (descriptive statistics).
- Read: SKS chapter 1 and 2.1; Kurose 1.1 and 1.5; Navidi 1.1 to 1.2.

KEY IDEA **Remember: store it, query it, keep it correct; and always trace the full path.**